

# DLL 만드는 방법

2023864046 하도윤

2026.10.15

# 목차




01 DLL 개념 설명

02 C 언어를 이용한 DLL 만들기

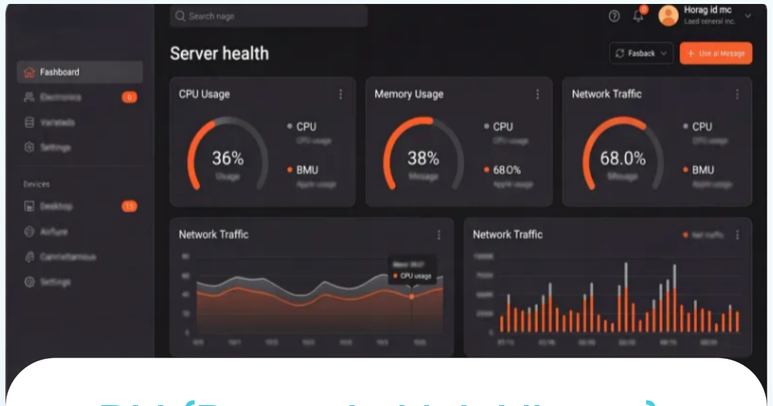
03 어셈블리어를 이용한 DLL 만들기

04 클라이언트 앱 만들기 및 실행



# DLL이란 무엇인가

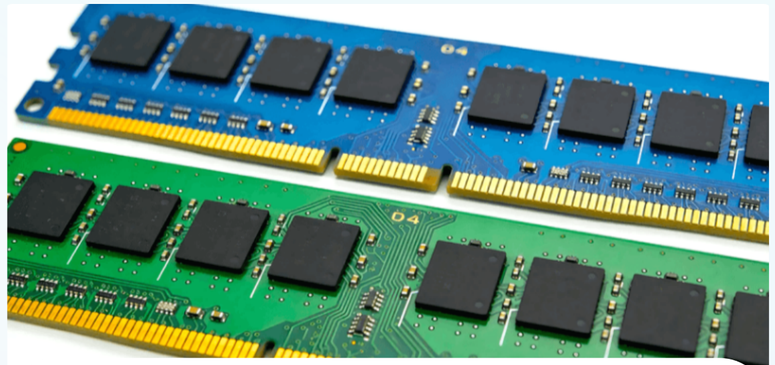
DLL(동적 연결 라이브러리)은 실행 시간에 프로그램에 연결되는 재사용 가능한 코드 라이브러리입니다. 여러 애플리케이션이 하나의 DLL을 공유할 수 있습니다.




## DLL(Dynamic Link Library) 동적 연결 라이브러리

- .dll 파일로 제공
- 코드, 데이터, 리소스로 구성
- 헤더 파일과 임포트 라이브러리 제공





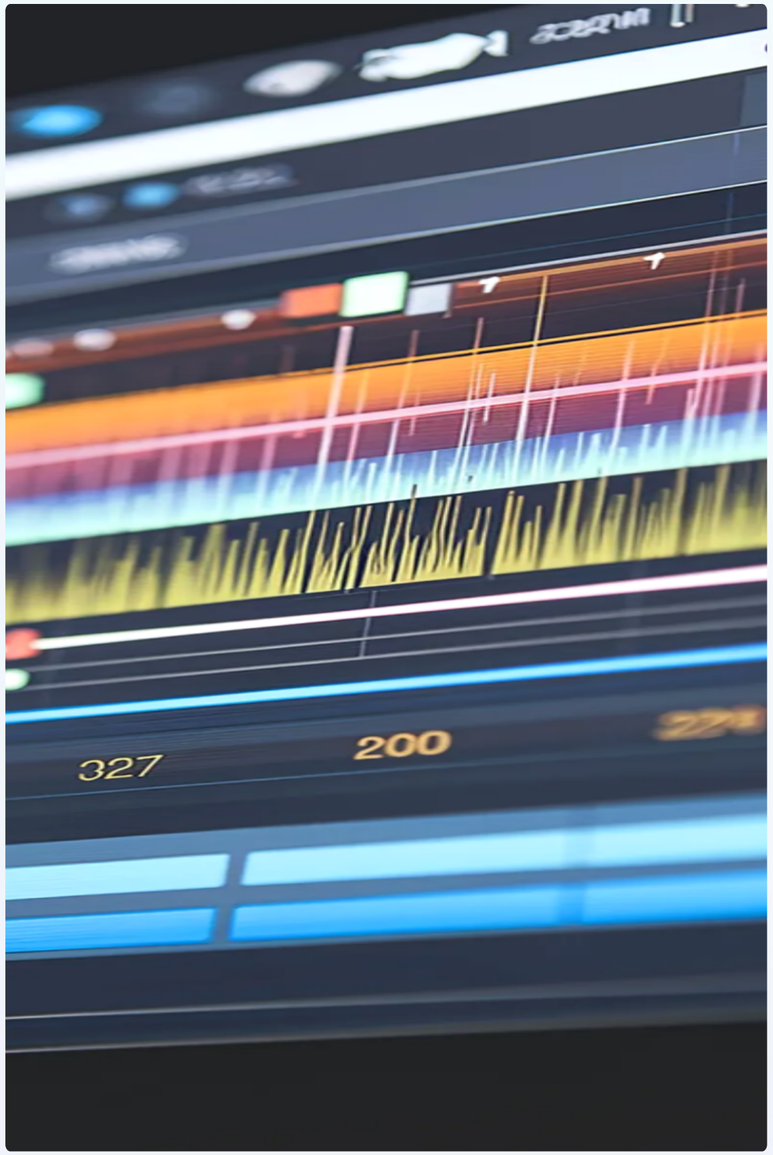
## LIB(Static Link Library) 정적 라이브러리와 비교

- 정적 라이브러리: 컴파일 시간에 연결
- 동적 라이브러리: 실행 시간에 연결
- DLL은 실행 파일 크기와 메모리 사용을 줄이고 교체만으로 업데이트 가능



# C 언어 DLL 구현 예제

헤더 파일은 DLL 외부에서 사용할 함수의 선언과 내보내기 매크로를 정의합니다.




## MathLibrary.h와 MathLibrary.c로 간단한 수학 함수를 DLL에 구현합니다.

MathLibrary.h에서 피보나치 수열 관련 함수를 선언하고, MathLibrary.c에서 피보나치 초기화, 다음 값 계산(오버플로우 체크 포함), 현재 값 및 인덱스 반환 함수 구현

### 함수 내보내기

__declspec(dllexport)는 함수를 외부에서 사용할 수 있도록 표시하고 내보내기 테이블에 추가합니다.

# C 언어 DLL 프로젝트 설정

**Step 1**

Step 1 DLL 프로젝트 생성

**Visual Studio 설치 설정 및 "언어","플랫폼","프로젝트 형식" 설정**

**Step 2**

Step 2 DLL 헤더 파일 작성

**외부에서 사용할 함수를 헤더 파일(.h)에 선언하고 __declspec(dllexport)로 내보냄**

**Step 3**

Step 3 DLL 소스 파일 작성

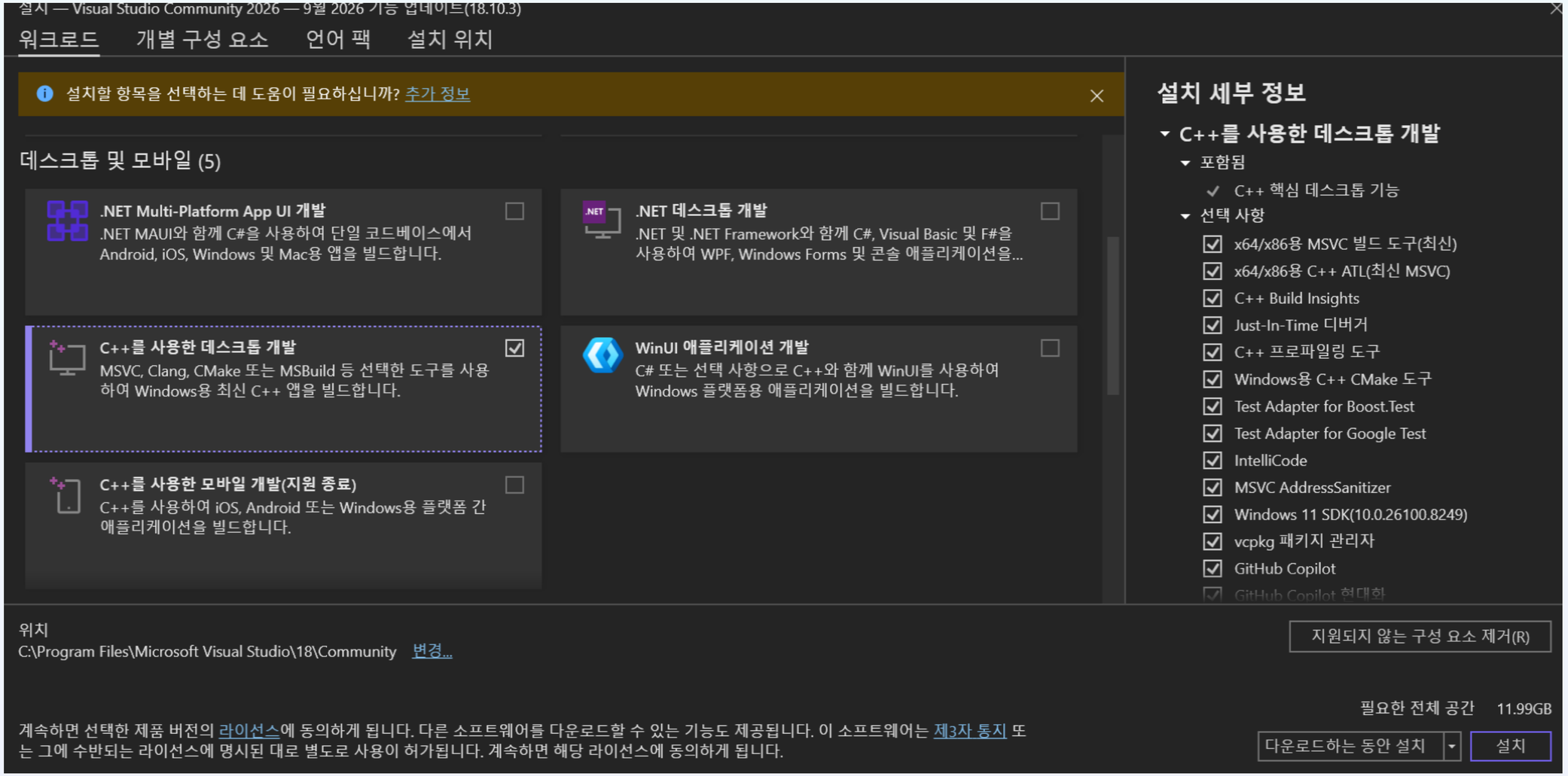
**함수의 실제 동작을 소스 파일(.c)에 구현**

**Step 4**

Step 4 DLL 프로젝트 빌드

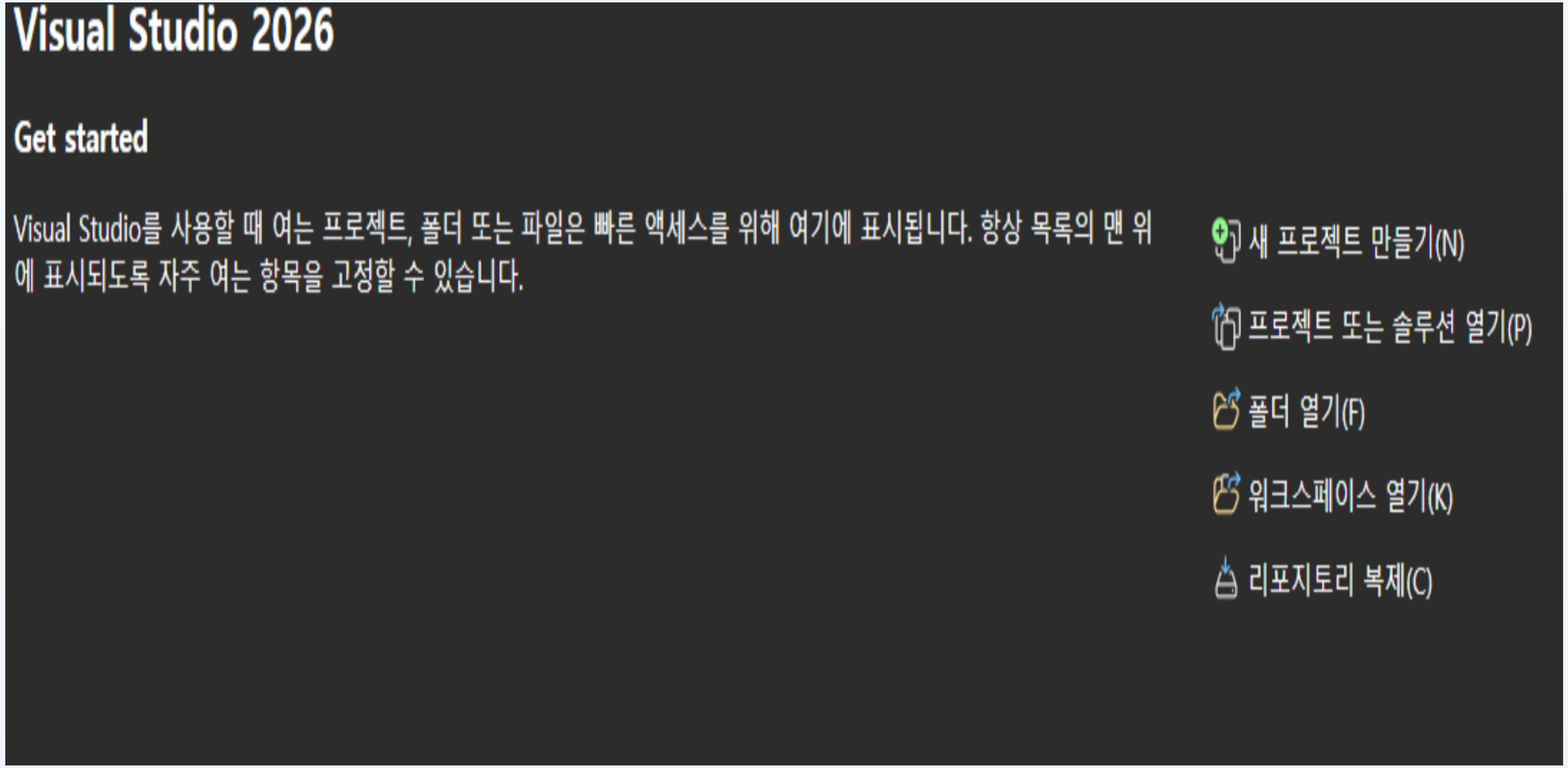
**빌드 구성을 Release로 설정하고 컴파일하면 .dll 파일이 생성**

# 1.DLL 프로젝트 생성




**Visual Studio는 C언어와 C++을 통합된 MSVC(Microsoft Visual C/C++)컴파일러 환경으로 제공합니다. 따라서 'C++를 사용한 데스크톱 개발' 워크로드를 설치해줍니다.**

# 1.DLL 프로젝트 생성




**새 프로젝트 만들기(N)를 선택하여 줍니다.**

# 1.DLL 프로젝트 생성

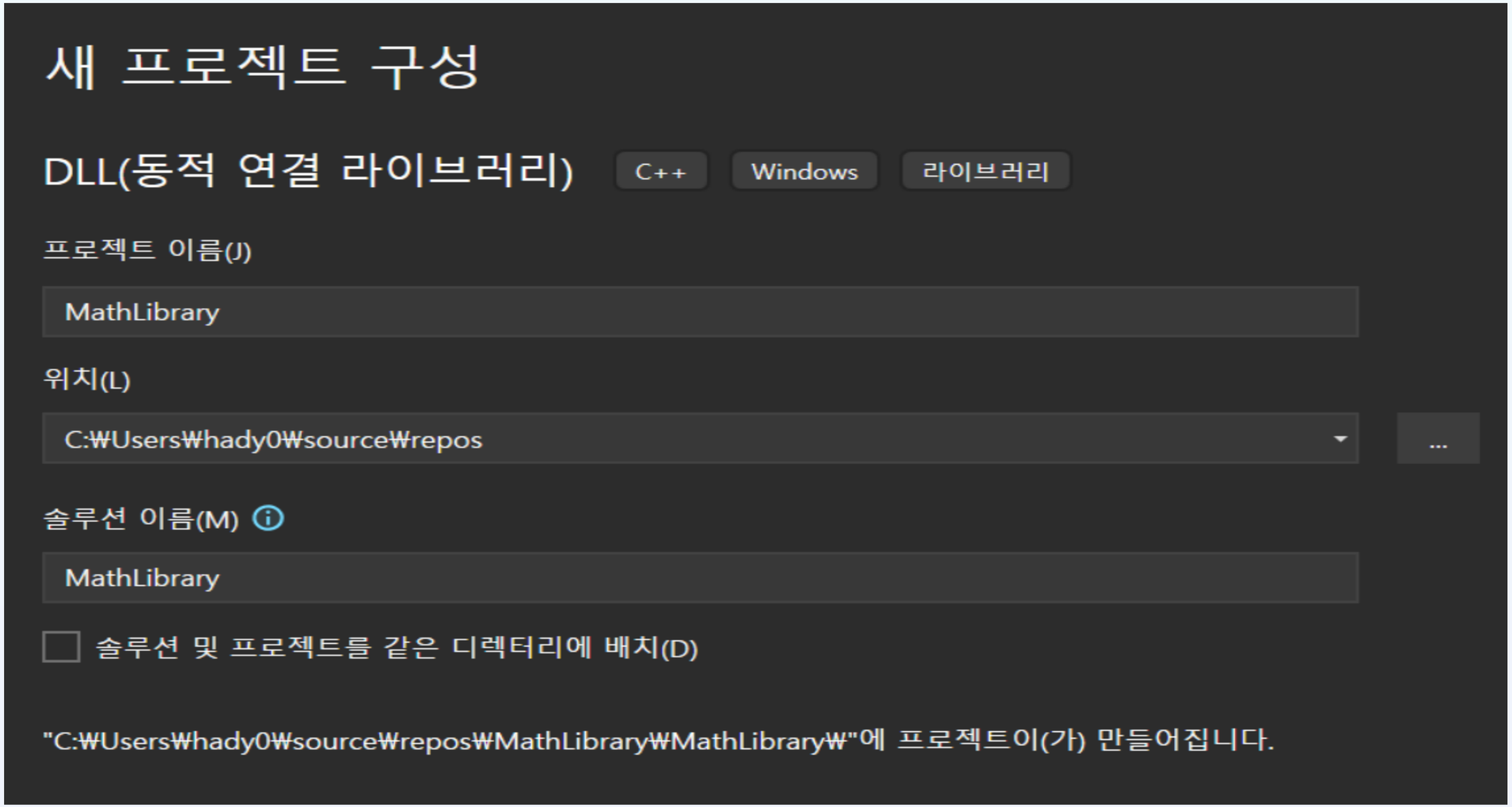






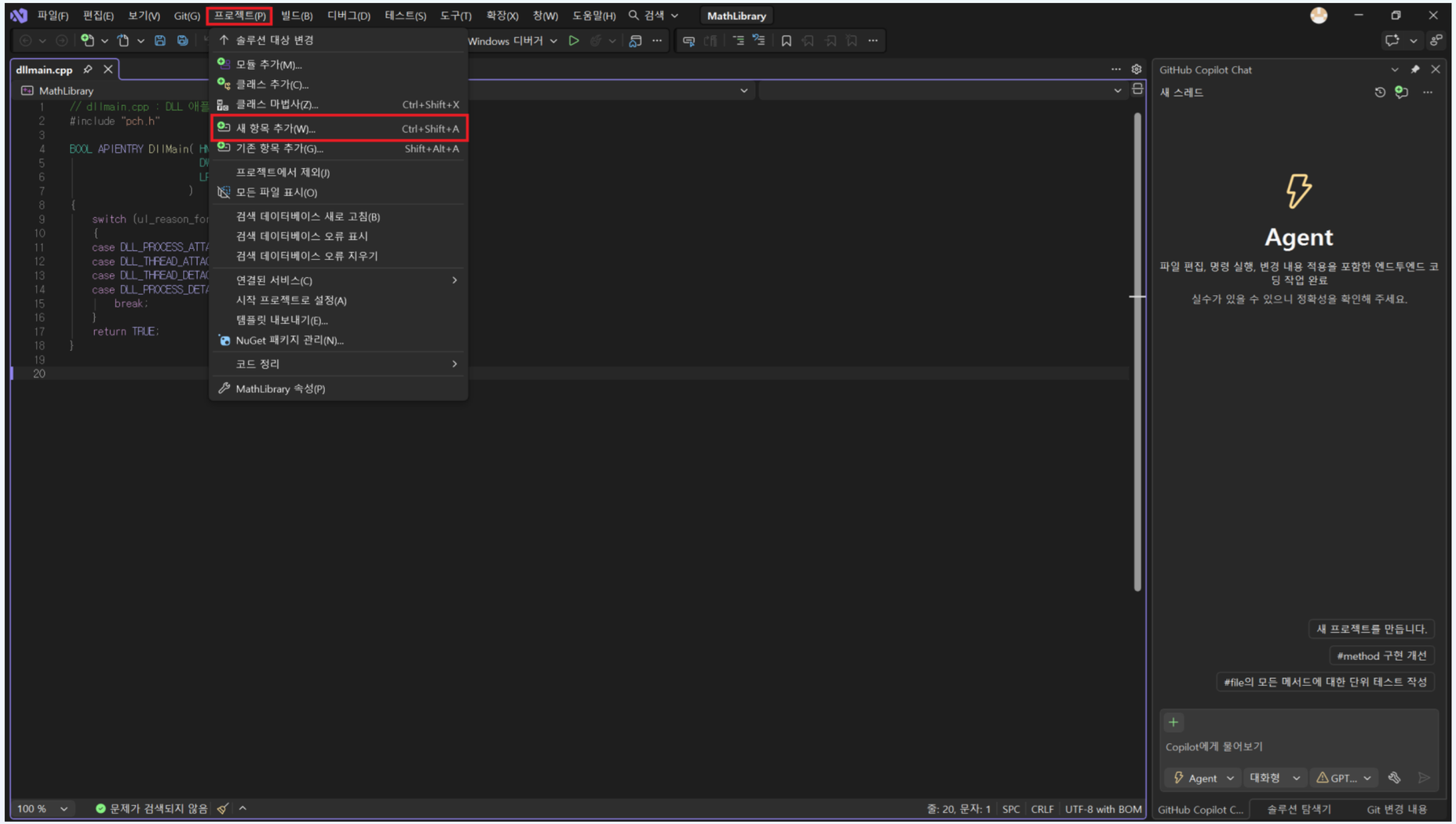
**모든 언어 -> C++, 모든 플랫폼 -> Windows, 모든 프로젝트 형식 -> 라이브러리(Library)로 설정하여줍니다.**

# 1.DLL 프로젝트 생성



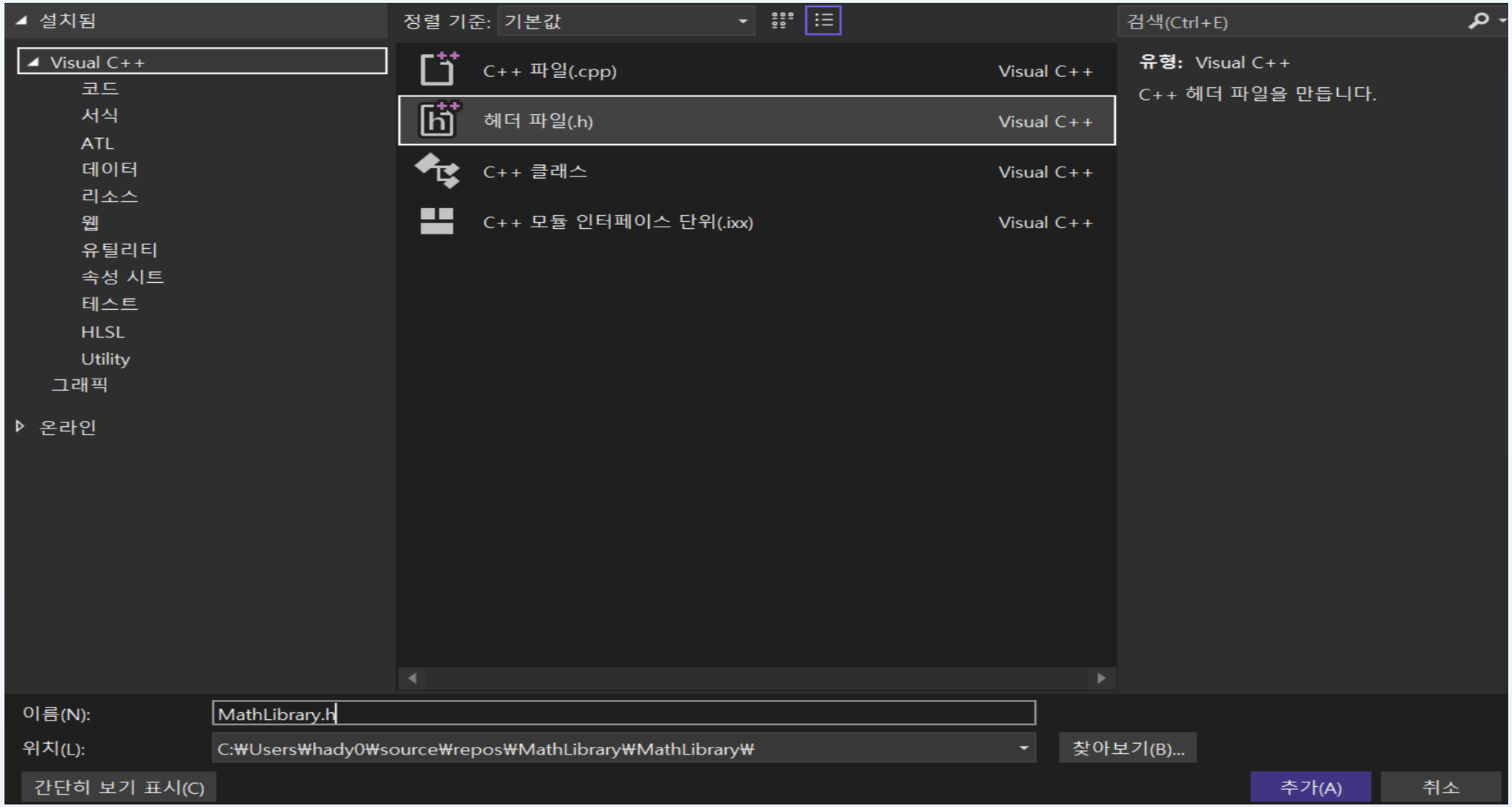

**프로젝트 이름을 MathLibrary로 바꿔줍니다.**

# 2. DLL 헤더 파일 작성




**프로젝트(P) 선택 후 새 항목 추가(W)를 선택해줍니다.**

# 2. DLL 헤더 파일 작성




**Visual C++에서 헤더 파일(h)를 선택하고 아래 이름을 MathLibrary.h로 바꿔준 후 추가를 눌러줍니다.**

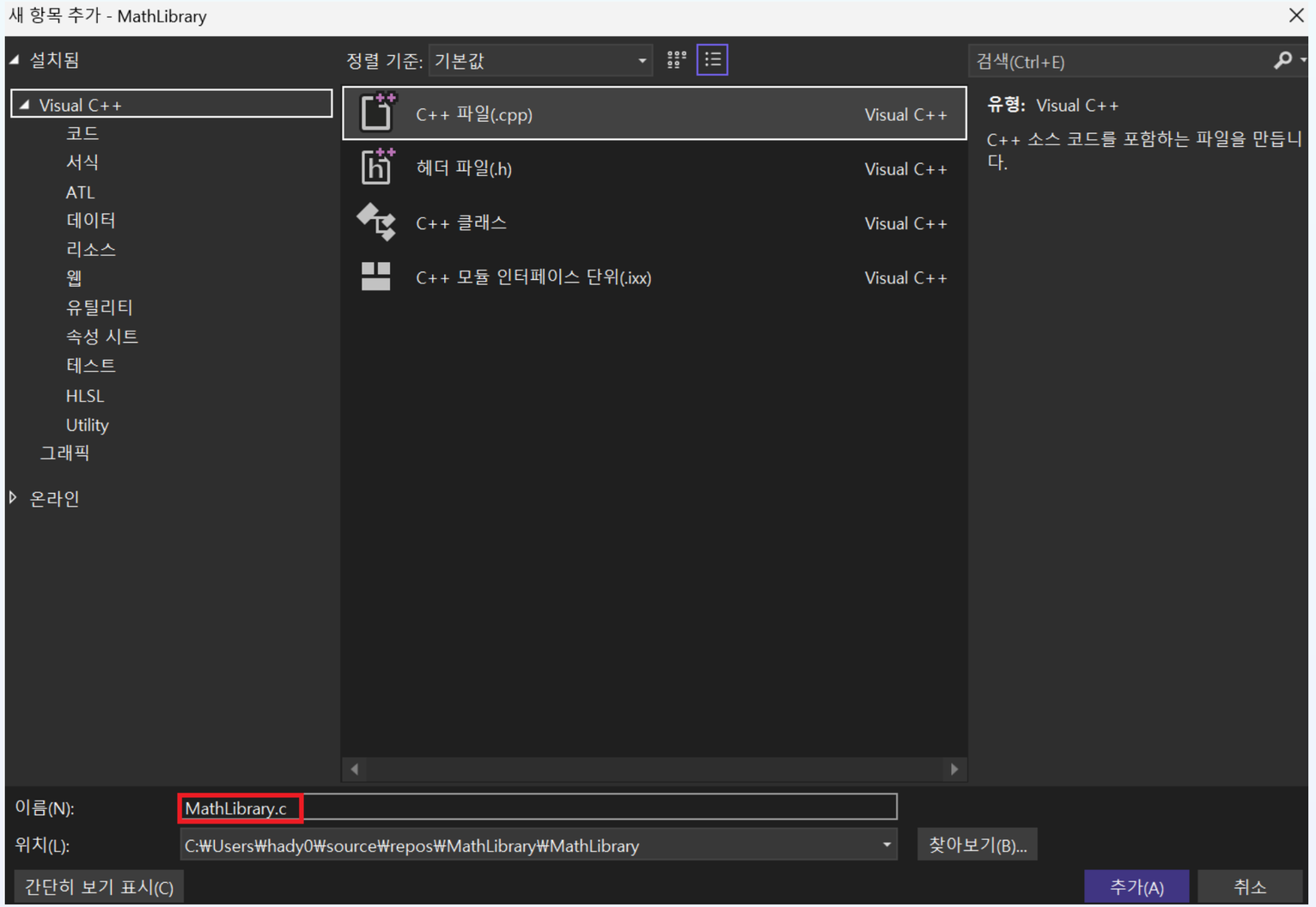
# 2. DLL 헤더 파일 작성

MathLibrary | (전역 범위) | fibonacci_index()

```
1   // MathLibrary.h - 수학 함수의 선언을 포함
2   #pragma once
3   #ifdef MATHLIBRARY_EXPORTS
4   // 함수들을 외부로 내보냄(Export)
5   #define MATHLIBRARY_API __declspec(dllexport)
6   #else
7   // DLL이 제공하는 함수들을 가져와서(Import) 사용할 수 있게 해줌
8   #define MATHLIBRARY_API __declspec(dllimport)
9   #endif
10
11  MATHLIBRARY_API long long MyAddFunction(long long a, long long b);
12
13  // 피보나치 점화식은 수열 F를 정의
14  // F(n)은 다음과 같다:
15  //              { n = 0 일 때, a
16  //              { n = 1 일 때, b
17  //              { n > 1 일 때, F(n-2) + F(n-1)
18  // 초기 정수 값 a와 b에 대해 적용
19  // 만약 F(0) = 1, F(1) = 1로 초기화된다면,
20  // 이 관계식은 잘 알려진 피보나치 수열을 생성: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
21
22  // F(0) = a, F(1) = b가 되도록 피보나치 수열 관계를 초기화
23  // 이 함수는 다른 어떤 함수보다 먼저 호출
24  MATHLIBRARY_API void fibonacci_init(
25      const unsigned long long a, const unsigned long long b);
26
27  // 수열의 다음 값을 생성
28  // 성공 시 1(true)을 반환하고 현재 값과 인덱스를 갱신
29  // 오버플로우 발생 시 0(false)을 반환하며 현재 값과 인덱스는 변경되지 않음
30  MATHLIBRARY_API int fibonacci_next();
31
32  // 수열의 현재 값을 가져옴
33  MATHLIBRARY_API unsigned long long fibonacci_current();
34
35  // 수열에서 현재 값의 위치(인덱스)를 가져옴
36  MATHLIBRARY_API unsigned fibonacci_index();
```

**마이크로소프트 도움말 사이트에서 제공하는 샘플 코드를 입력해줍니다.**

# 3.DLL 소스 파일 작성




**C언어를 기준으로 만들기 때문에 소스 파일 이름을 MathLibrary.c로 만들어줍니다.**

# 3. DLL 소스 파일 작성

MathLibrary.c* | MathLibrary.h*

MathLibrary | (전역 범위) | fibonacci_index()

```
// MathLibrary.c : Defines the exported functions for the DLL.
#include "pch.h" // Visual Studio 2017 이전 버전은 "stdafx.h" 사용
#include <limits.h>
#include "MathLibrary.h"

// DLL 내부 상태 변수:
static unsigned long long previous_;  // 이전 값
static unsigned long long current_;   // 현재 수열 값
static unsigned index_;               // 현재 수열 위치

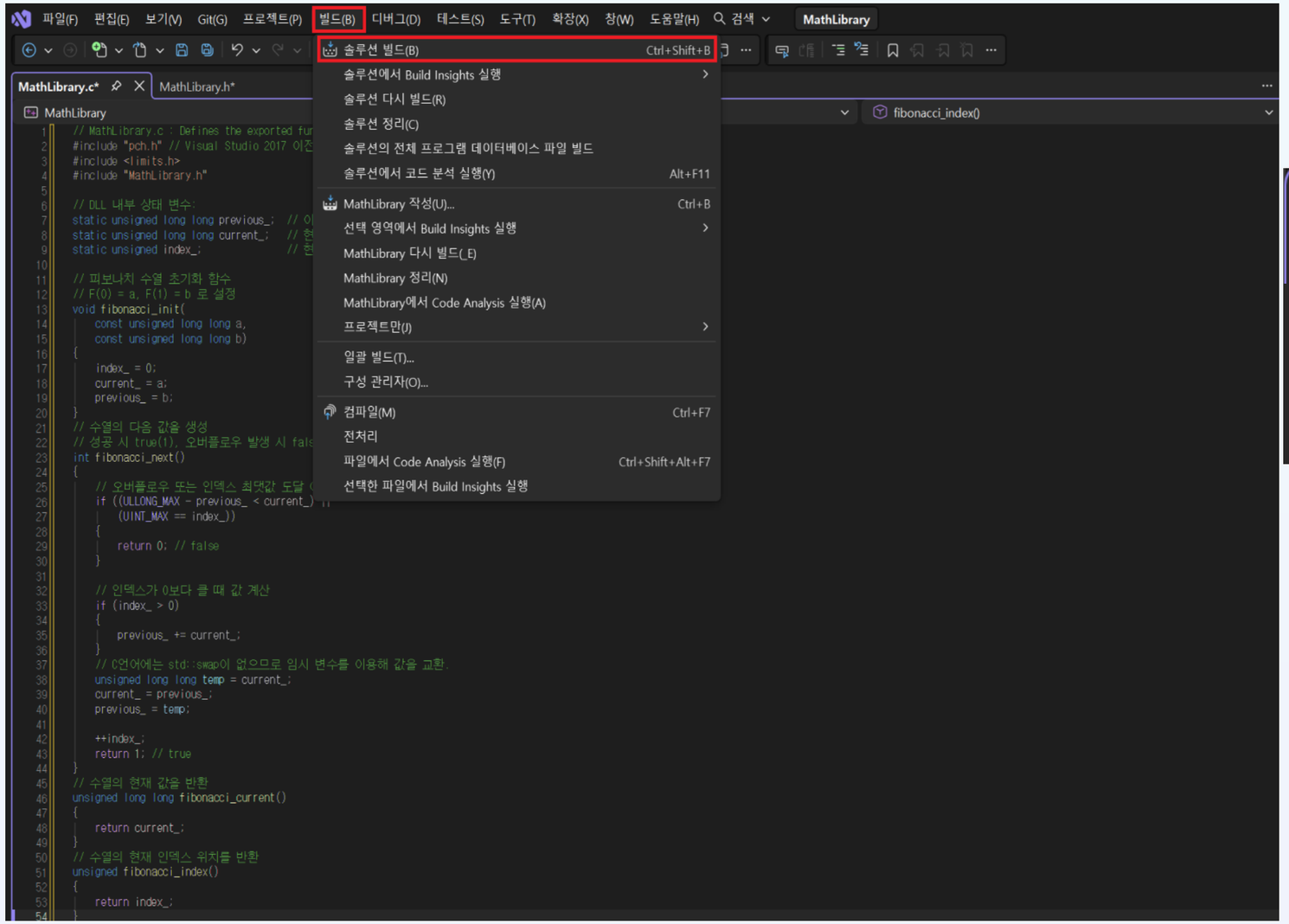
// 피보나치 수열 초기화 함수
// F(0) = a, F(1) = b 로 설정
void fibonacci_init(
    const unsigned long long a,
    const unsigned long long b)
{
    index_ = 0;
    current_ = a;
    previous_ = b;
}
// 수열의 다음 값을 생성
// 성공 시 true(1), 오버플로우 발생 시 false(0) 반환
int fibonacci_next()
{
    // 오버플로우 또는 인덱스 최댓값 도달 여부 확인
    if ((ULLONG_MAX - previous_ < current_) ||
        (UINT_MAX == index_))
    {
        return 0; // false
    }

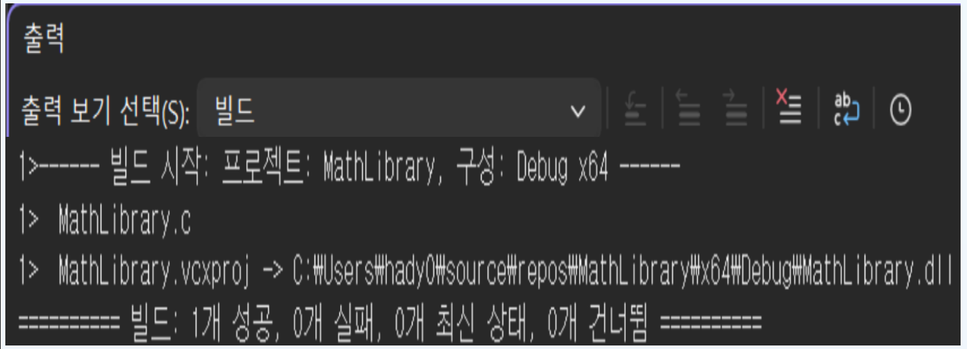
    // 인덱스가 0보다 클 때 값 계산
    if (index_ > 0)
    {
        previous_ += current_;
    }
    // C언어에는 std::swap이 없으므로 임시 변수를 이용해 값을 교환.
    unsigned long long temp = current_;
    current_ = previous_;
    previous_ = temp;

    ++index_;
    return 1; // true
}
// 수열의 현재 값을 반환
unsigned long long fibonacci_current()
{
    return current_;
}
// 수열의 현재 인덱스 위치를 반환
unsigned fibonacci_index()
{
    return index_;
}
```

**마이크로소프트 도움말 사이트에서 제공하는 샘플 코드를 입력해줍니다.**

# 4. DLL 프로젝트 빌드




**상단에 빌드(B)를 선택해준 후 솔루션 빌드(B)를 클릭하면 오른쪽과 같이 출력됩니다.**



# 어셈블리어 DLL 구현 예제

MASM 소스에서 덧셈 함수를 구현합니다. 함수는 레지스터의 매개변수를 읽고 rax 레지스터에 연산 결과를 저장한 뒤 반환합니다.

## MASM으로 수학 함수를 포함한 DLL 작성

### 어셈블리 소스 파일

MathLib.asm에 PUBLIC 선언 후 각 함수는 두 정수를 받아 결과를 반환합니다.

### 호출 규약 이해

스택을 거치지 않고 레지스터를 통해 인자를 전달받습니다.
첫 번째 인자는 RCX, 두 번째 인자는 RDX로 곧바로 전달됩니다.

### 레지스터 사용

첫 번째 매개변수는 RCX, 두 번째 매개변수는 RDX에 들어 있는 상태로 시작합니다.
코드(mov rax, rcx, add rax, rdx)처럼 레지스터를 직접 더한 뒤, 결과는 RAX 레지스터에 담아 반환(ret)합니다.

# 어셈블리어 DLL 만들기

**Step 1**

Step 1 프로젝트 설정

**어셈블리어 소스 파일(.asm) 컴파일을 위한 MASM 설정**

**Step 2**

Step 2 소스 파일 작성

**함수의 실제 동작을 .asm 파일에 구현**

**Step 3**

Step 3 헤더 파일 코드 추가 및 DLL 프로젝트 빌드

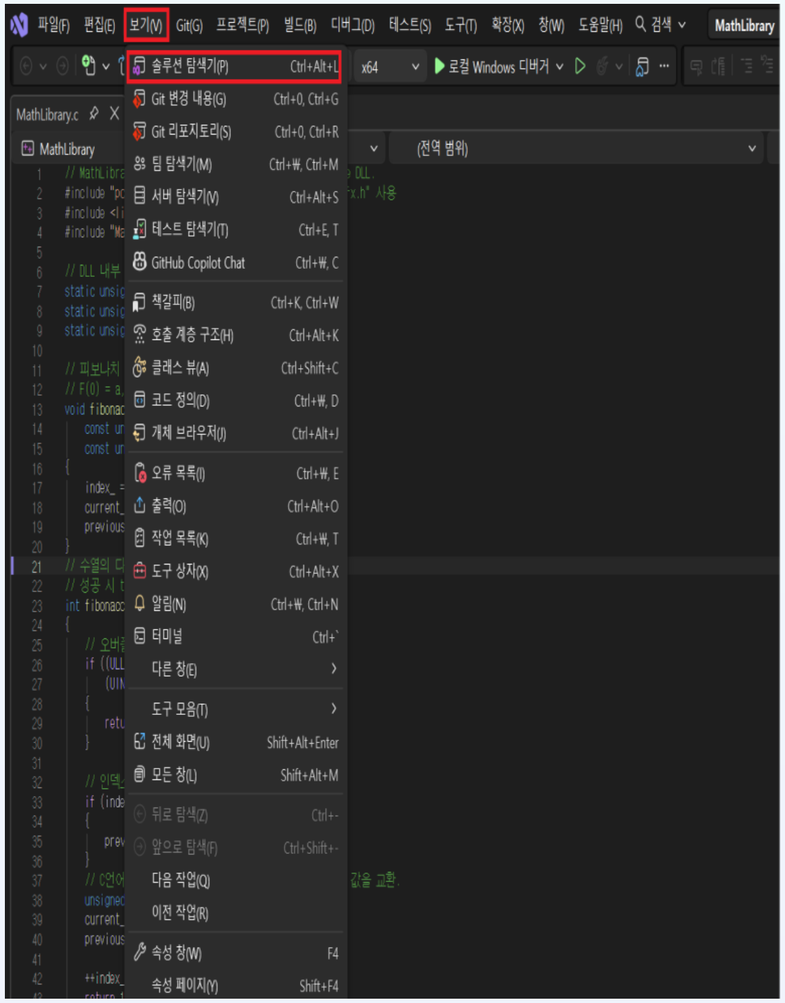
**MATHLIBRARY_API long long MyAddFunction(long long a, long long b); 코드 추가 및 빌드 구성을 Release로 설정하고 컴파일하면 .dll 파일이 생성**

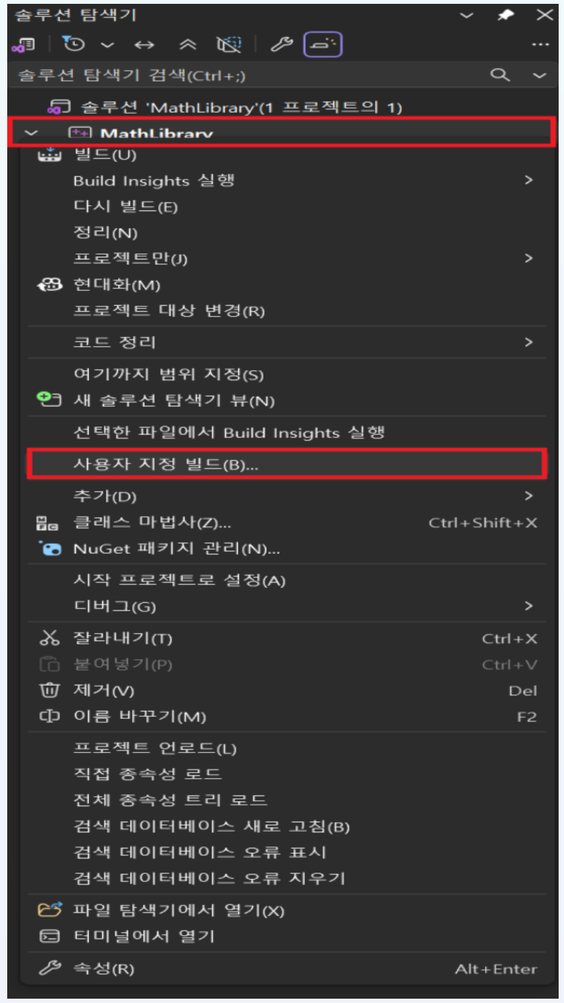
**Step 4**

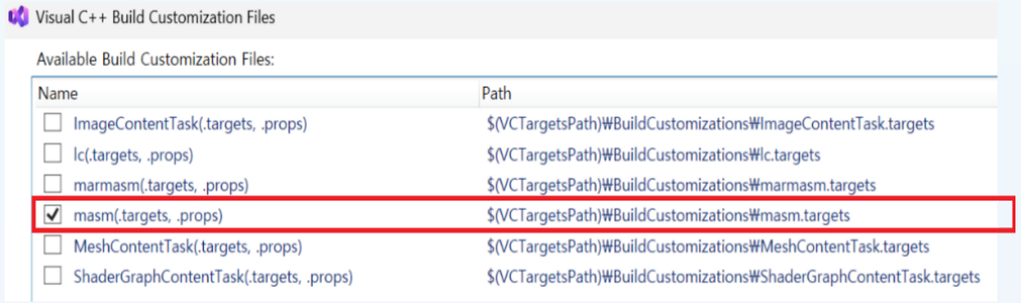
Step 4 .def 파일 생성

**모듈 정의 파일(.def)을 사용하여 함수를 내보내도록 설정**

# 1. 프로젝트 설정

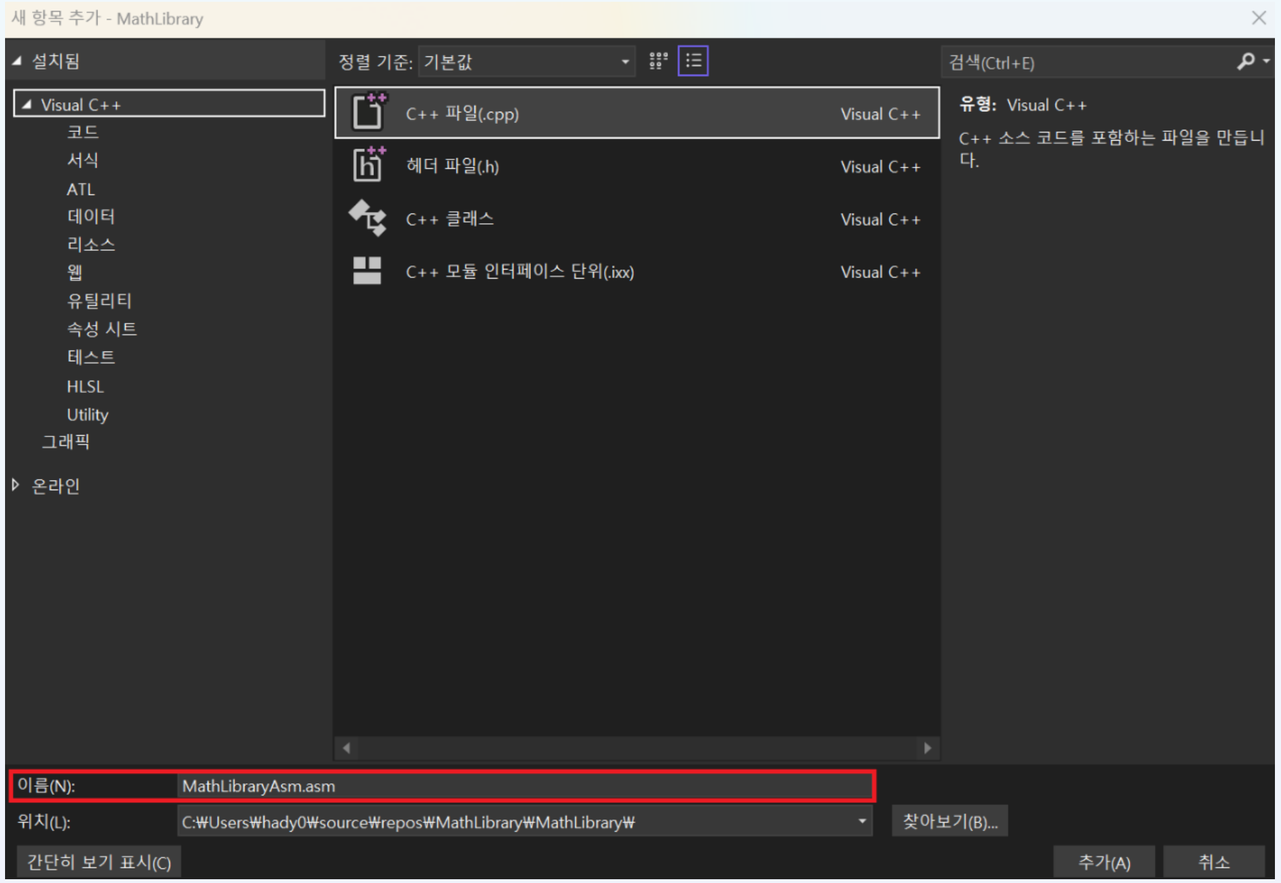



**솔루션 탐색기를 통해 프로젝트에서 사용자 지정 빌드에 들어가 MASM를 선택하여줍니다.**

# 2. 소스 파일 작성

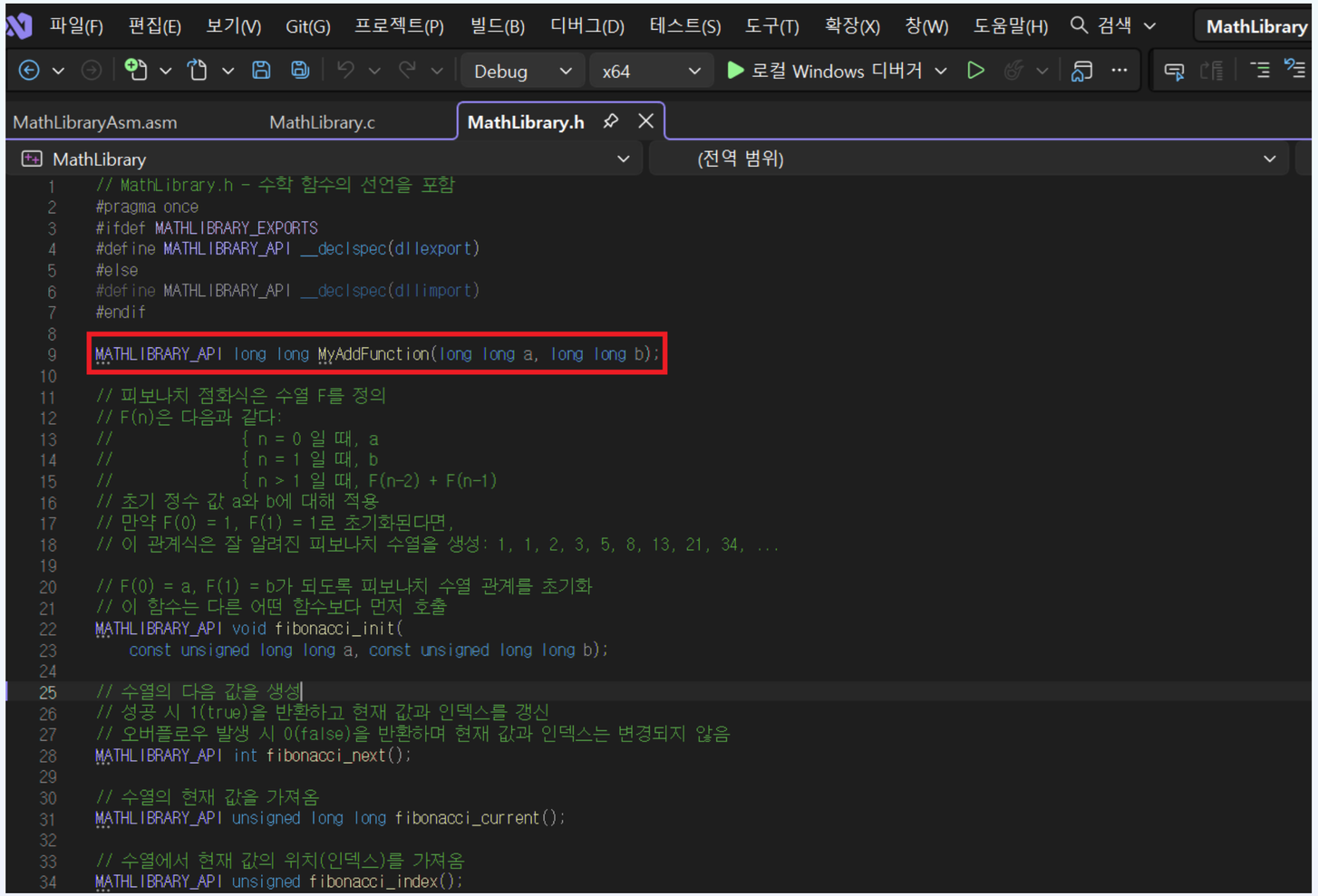



MathLibraryAsm.asm

```
1   .code
2
3   PUBLIC MyAddFunction  ; 외부(C언어 파일)에서 이 함수를 찾을 수 있도록 공개 선언
4   ; MATHLIBRARY_API long long MyAddFunction(long long a, long long b);
5   ; RCX = a, RDX = b
6   MyAddFunction proc
7       mov rax, rcx        ; RAX에 첫 번째 매개변수(a)를 대입
8       add rax, rdx        ; RAX = a + b (두 번째 매개변수 더하기)
9       ret                 ; RAX에 담긴 64비트 결과값이 C 환경으로 반환됨
10  MyAddFunction endp
11
12  end
```

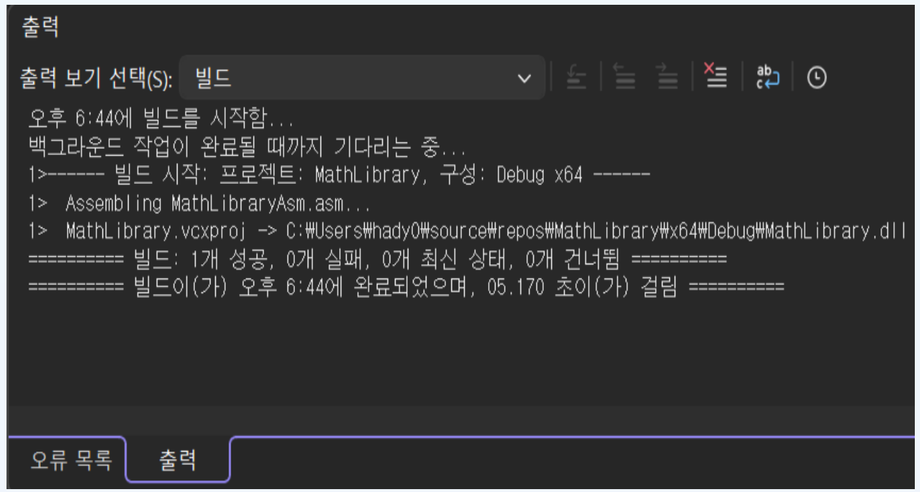
**MathLibraryAsm.asm 이름의 소스 파일 생성 후  코드를 작성하여 줍니다.**

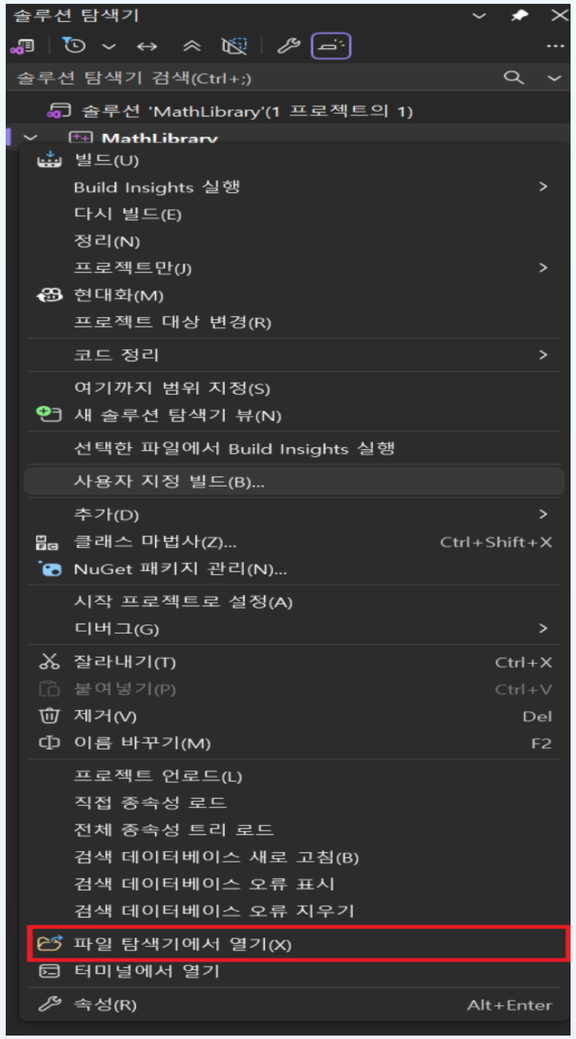
# 3. 헤더 파일 코드 추가




**어셈블리 코드로 작성된 함수를 C 언어 프로젝트에서 인식할 수 있게 하고, 외부에 DLL 함수로 공개(Export)하기 위해 MATHLIBRARY_API long long MyAddFunction(long long a, long long b); 코드 추가**

# 3. DLL 프로젝트 빌드



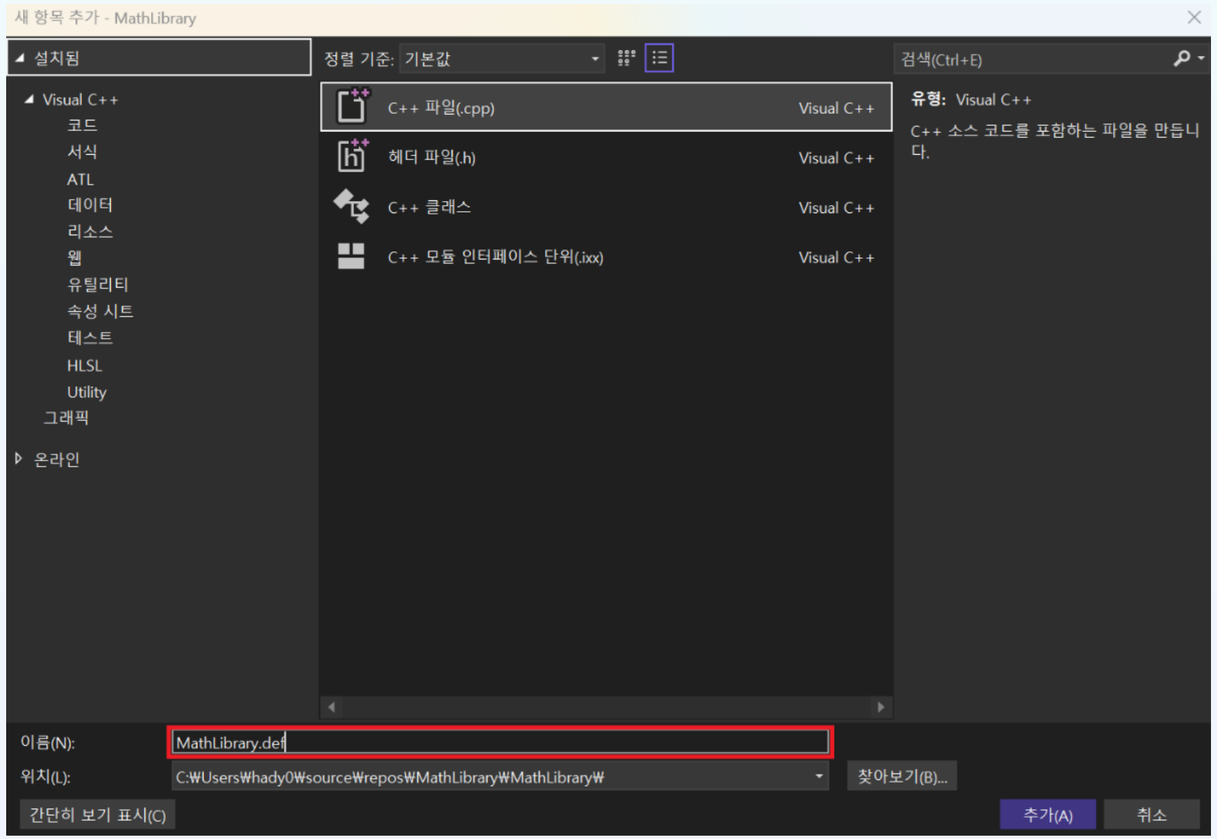



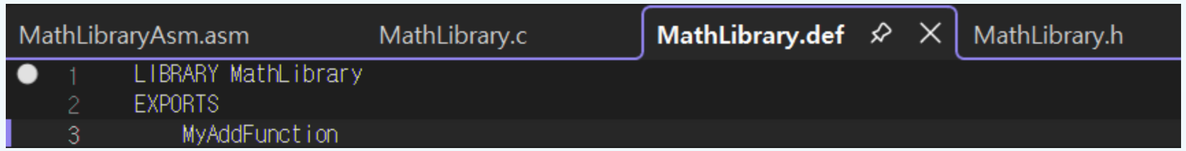


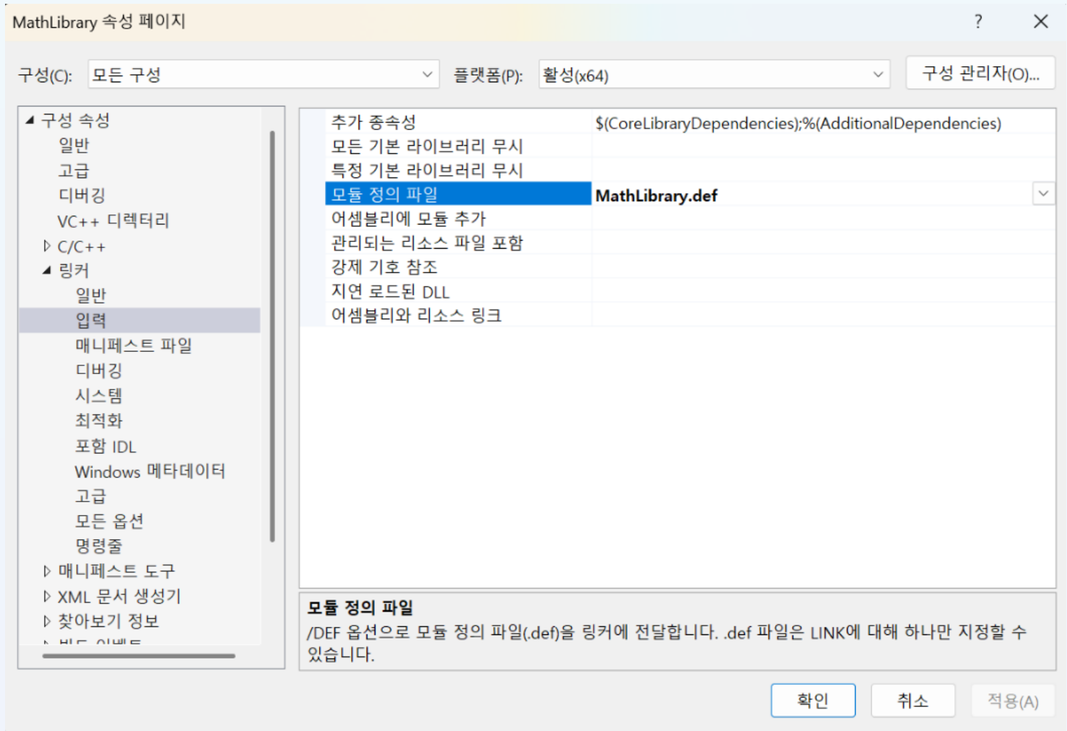


**빌드를 실행한 후 파일 탐색기를 열어보면 .dll 파일과 .lib 파일이 생성된걸 확인할 수 있습니다.**

# 4. .def 파일 생성




**C언어 함수는 __declspec(dllexport)를 붙여 내보낼 수 있지만, 어셈블리어 파일(.asm)은 이 키워드를 직접 사용할 수 없습니다. 따라서 모듈 정의 파일(.def)을 사용하여 함수를 내보내도록 설정해주어야 합니다.**

# 클라이언트 앱 만들기

**Step 1**

Step 1 클라이언트 앱 프로젝트 생성
콘솔 앱 만들기

**Step 2**

Step 2 포함 경로에 DLL 헤더 파일 추가 및 클라이언트 소스 파일 코드 작성
클라이언트 원본 파일에서 DLL 헤더 파일이 포함된 파일로의 상대 경로 입력

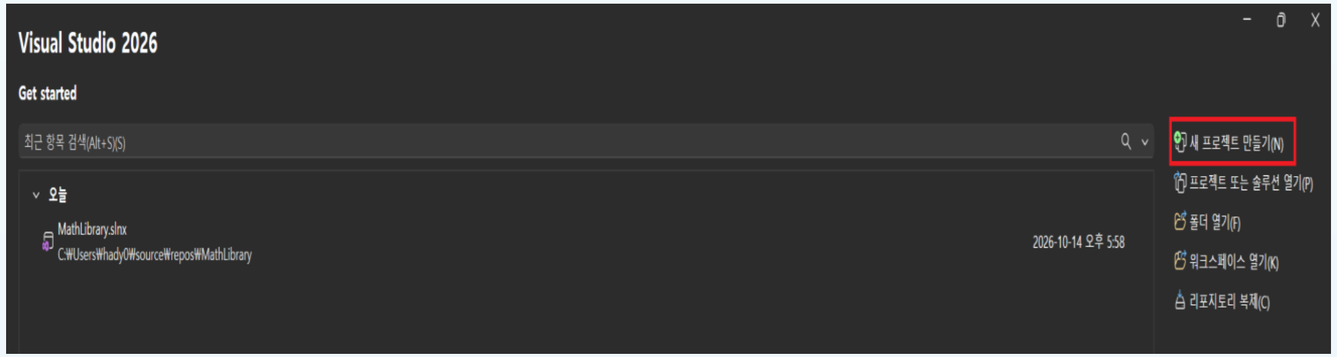
**Step 3**

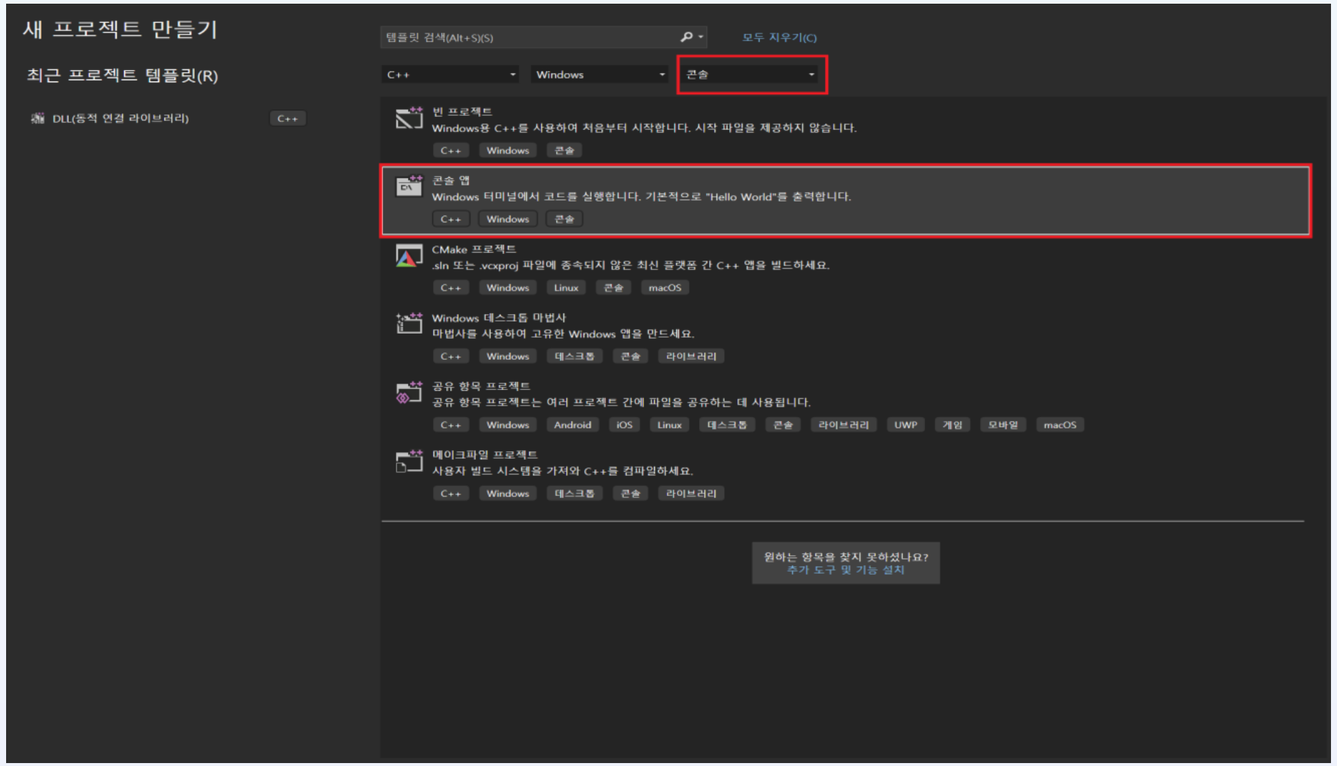
Step 3 프로젝트에 DLL 가져오기
상대 경로 매크로 방식 설정 및 ..₩..₩MathLibrary₩$(IntDir) 명령어 입력

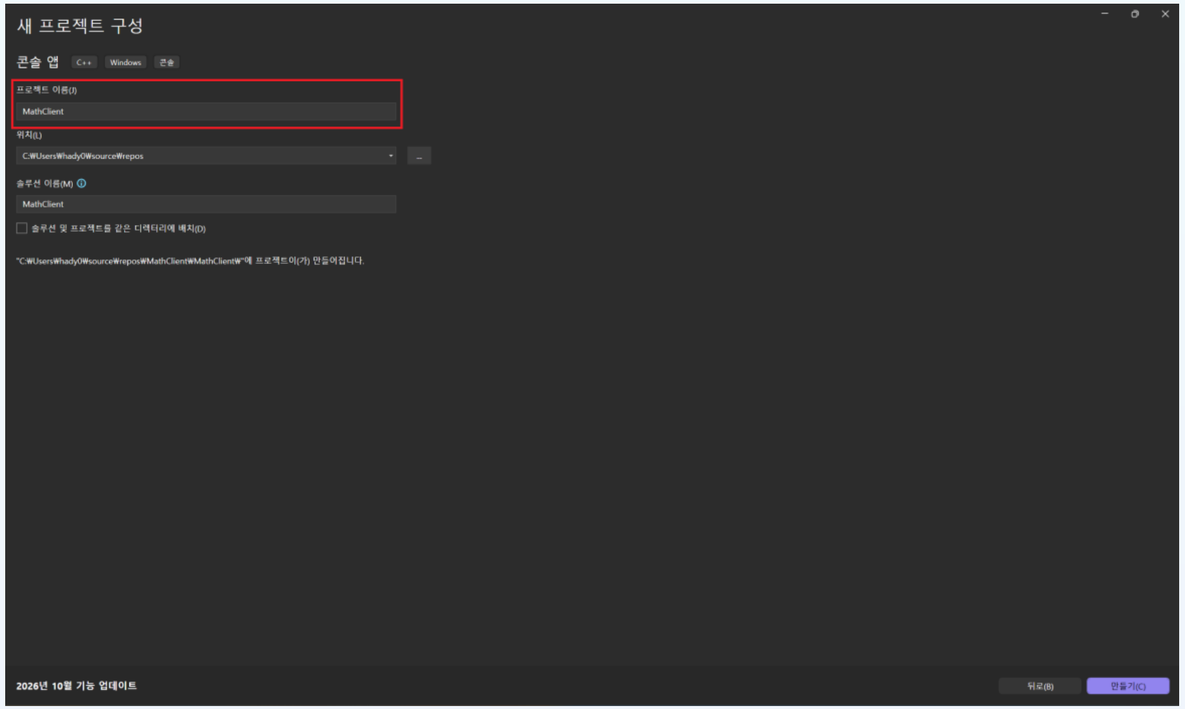
**Step 4**

Step 4 빌드 후 이벤트 설정 및 클라이언트 앱 실행
xcopy /y /d "..₩..₩MathLibrary₩$(IntDir)MathLibrary.dll" "$(OutDir)" 명령어 입력

# 1. 클라이언트 앱 프로젝트 생성



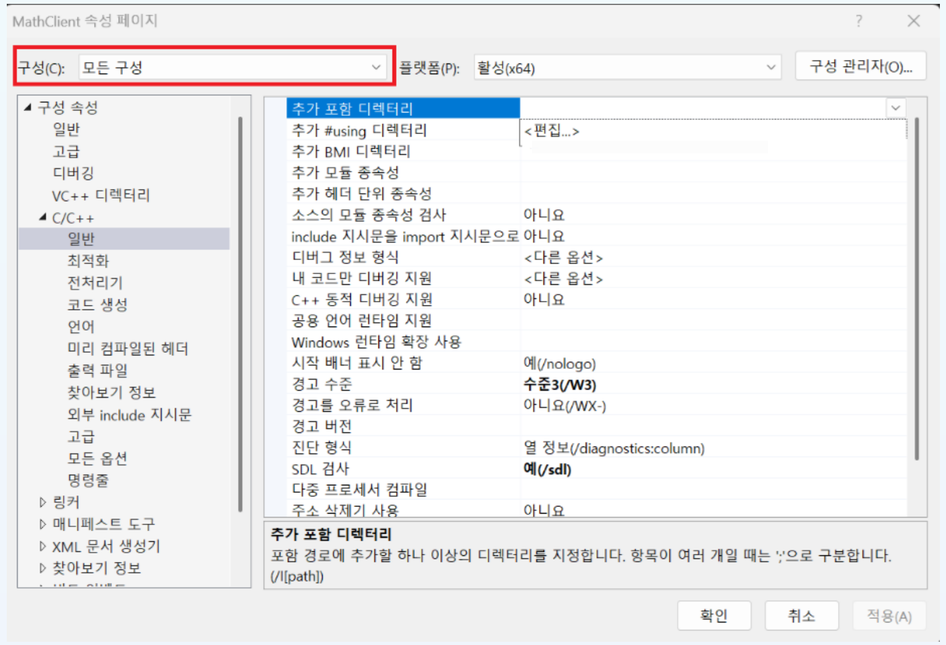



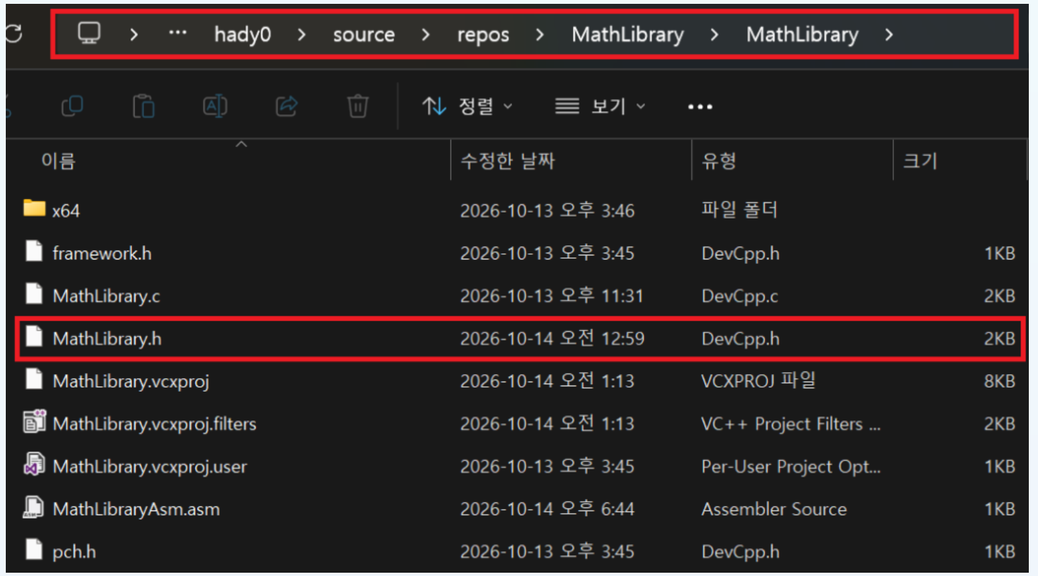



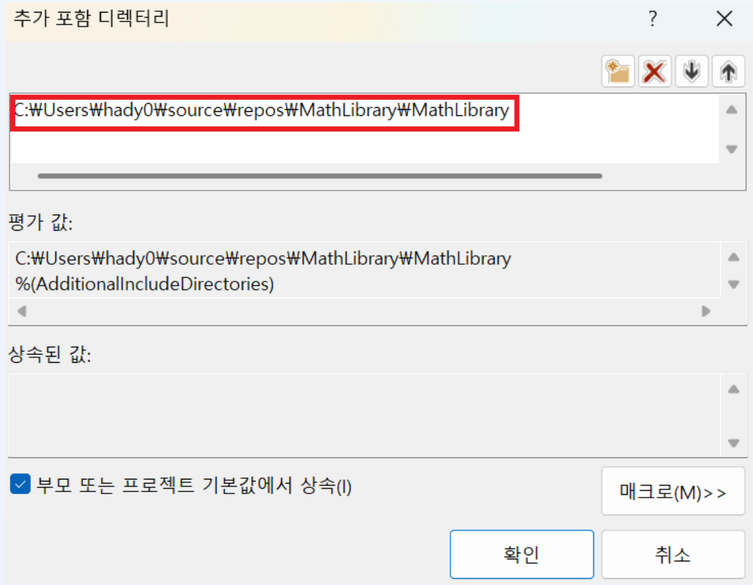

**프로젝트 형식을 콘솔**로 설정하고
콘솔 앱으로 MathClient라는 이름의
프로젝트를 만들어줍니다.

# 2. 포함 경로에 DLL 헤더 파일 추가




**클라이언트 앱 프로젝트 속성에서 헤더 파일(MathLibrary.h)이 들어있는 파일 경로를 추가 포함 디렉터리에 설정하여 줍니다.**

# 2. 클라이언트 소스 파일 코드 작성

MathClient.c

MathClient | (전역 범위) | main()

```c
#include <stdio.h>
#include "MathLibrary.h"

// 어셈블리 함수 선언
__declspec(dllimport) long long MyAddFunction(long long a, long long b);

int main()
{
    // 1. 피보나치 관계 수열 초기화 및 출력
    fibonacci_init(1, 1);

    // 오버플로우가 발생할 때까지 수열 값 출력
    do {
        printf("%u: %llu\n", fibonacci_index(), fibonacci_current());
    } while (fibonacci_next());

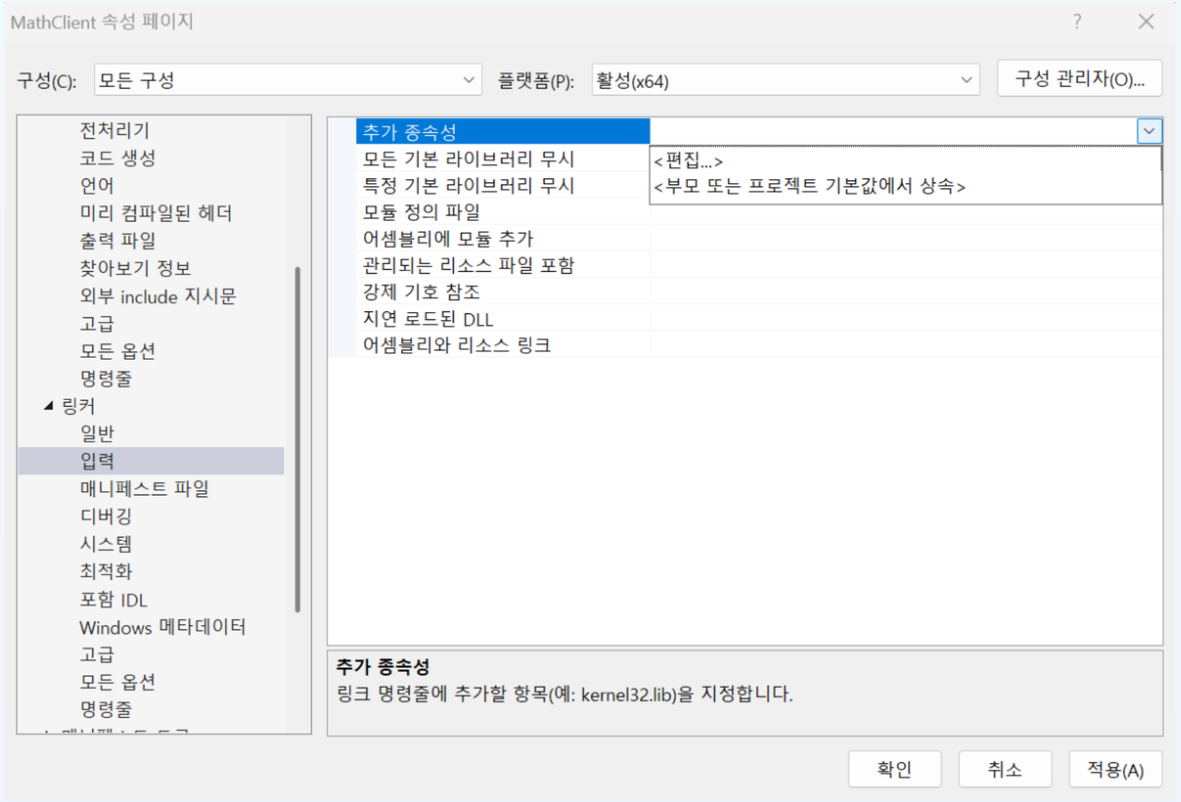
    // 오버플로우 전에 기록된 값의 개수 출력
    printf("%u Fibonacci sequence values fit in an unsigned 64-bit integer.\n\n", fibonacci_index() + 1);

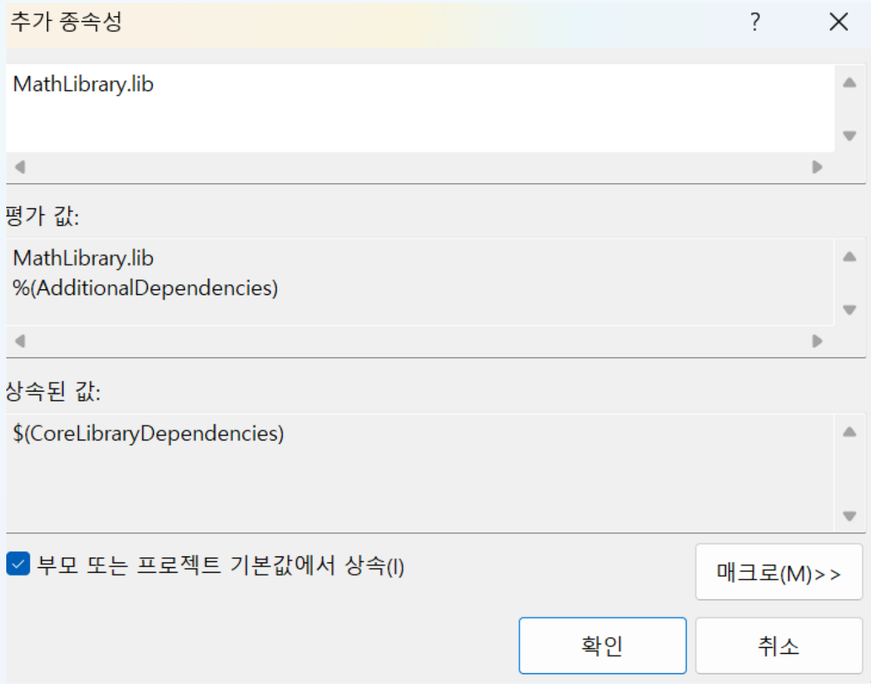
    // 2. 어셈블리로 만든 MyAddFunction 테스트
    long long result = MyAddFunction(12345, 67890);
    printf("Assembly MyAddFunction Result (12345 + 67890) = %lld\n", result);

    return 0;
}
```

C언어로 작성한 피보나치 수열이 끝까지 출력된 뒤 어셈블리로 작성한 덧셈 결과까지 깔끔하게 출력되기 위해 __declspec(dllimport) long long MyAddFunction(long long a, long long b); 코드를 입력하여 어셈블리 함수를 선언해줍니다.

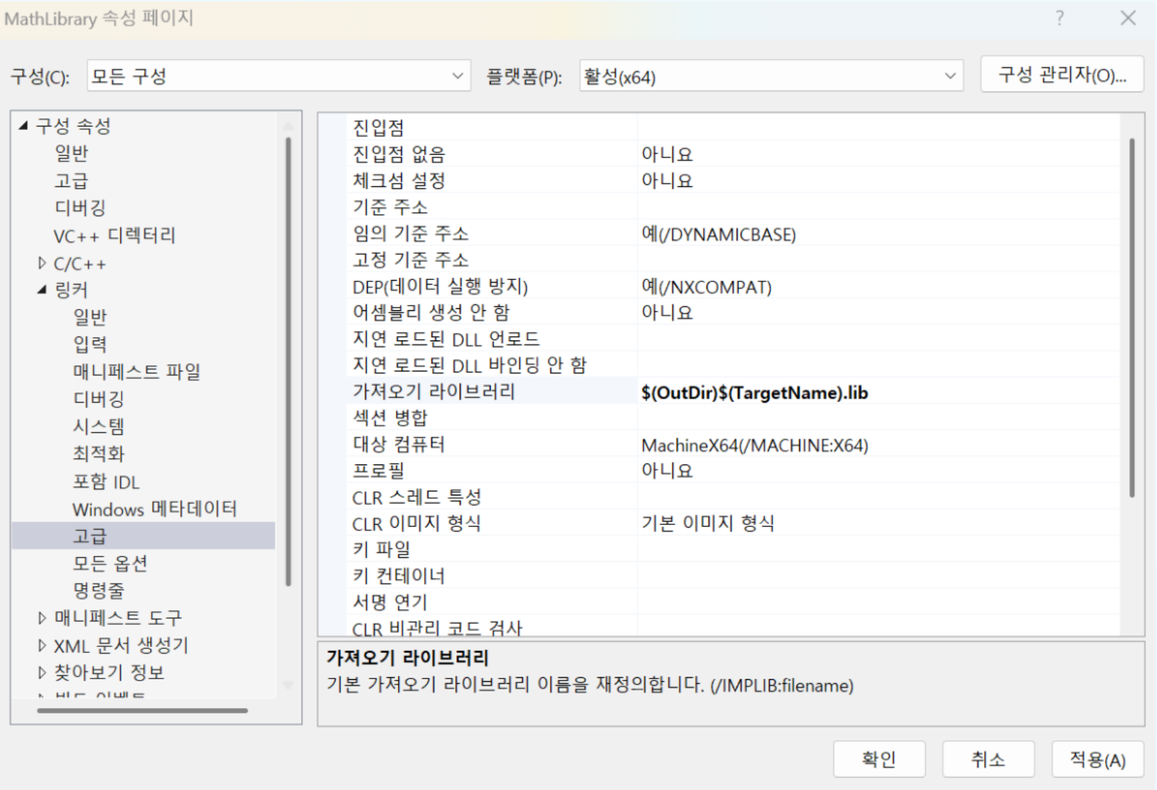
# 3. 프로젝트에 DLL 가져오기

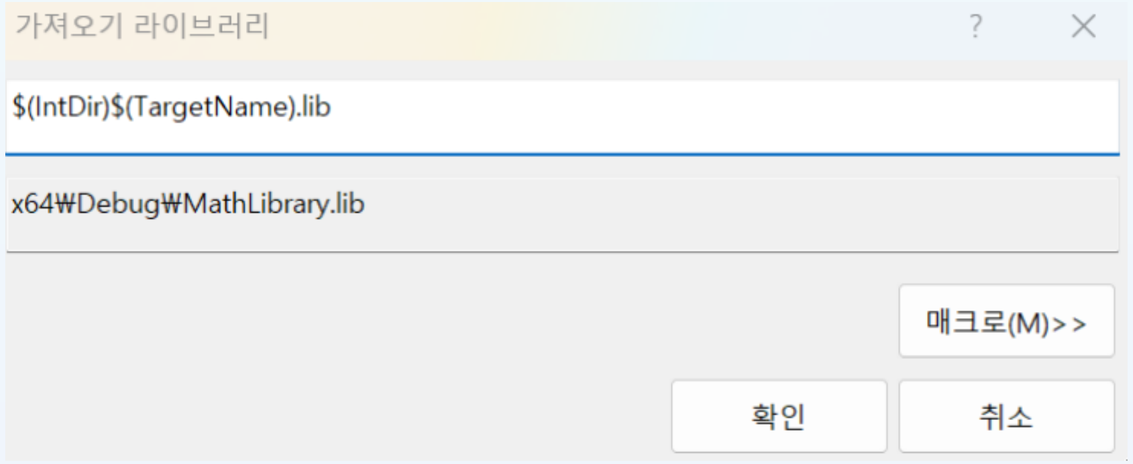



**추가 종속성 편집에서 MathLibrary.lib를 입력하여줍니다.**

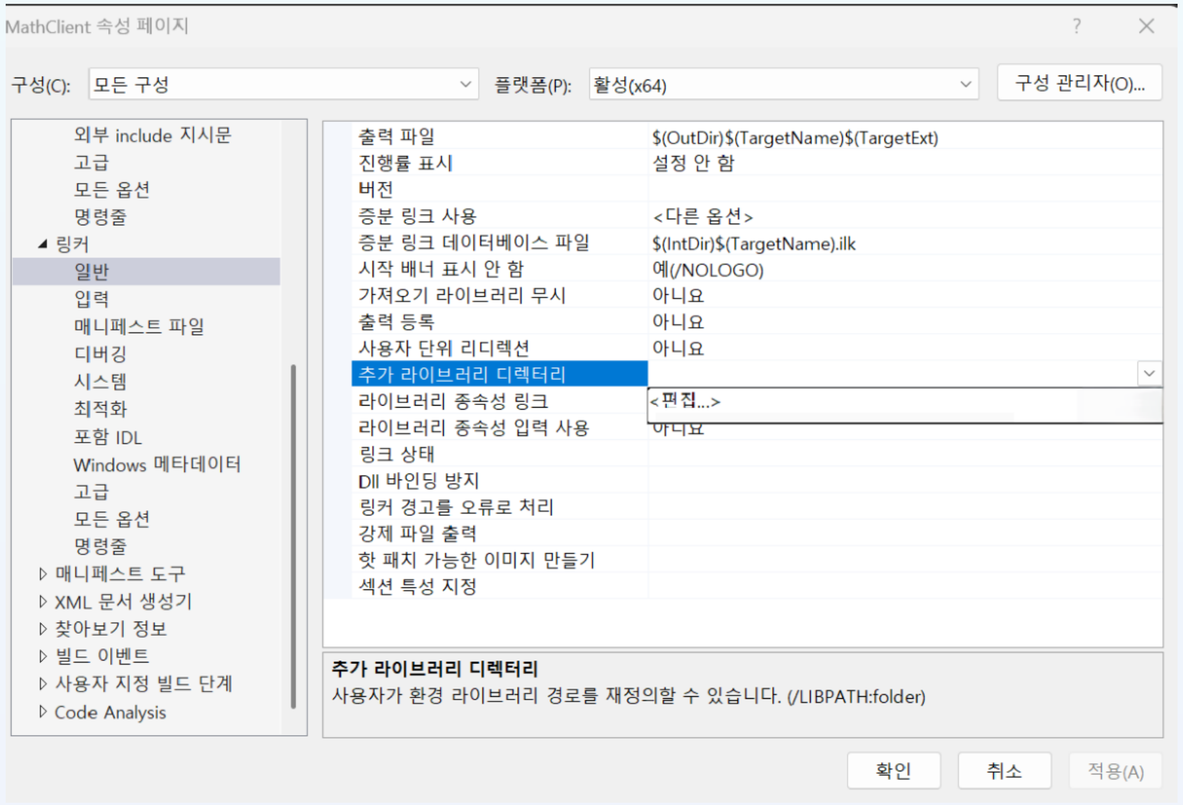
# 3. 프로젝트에 DLL 가져오기

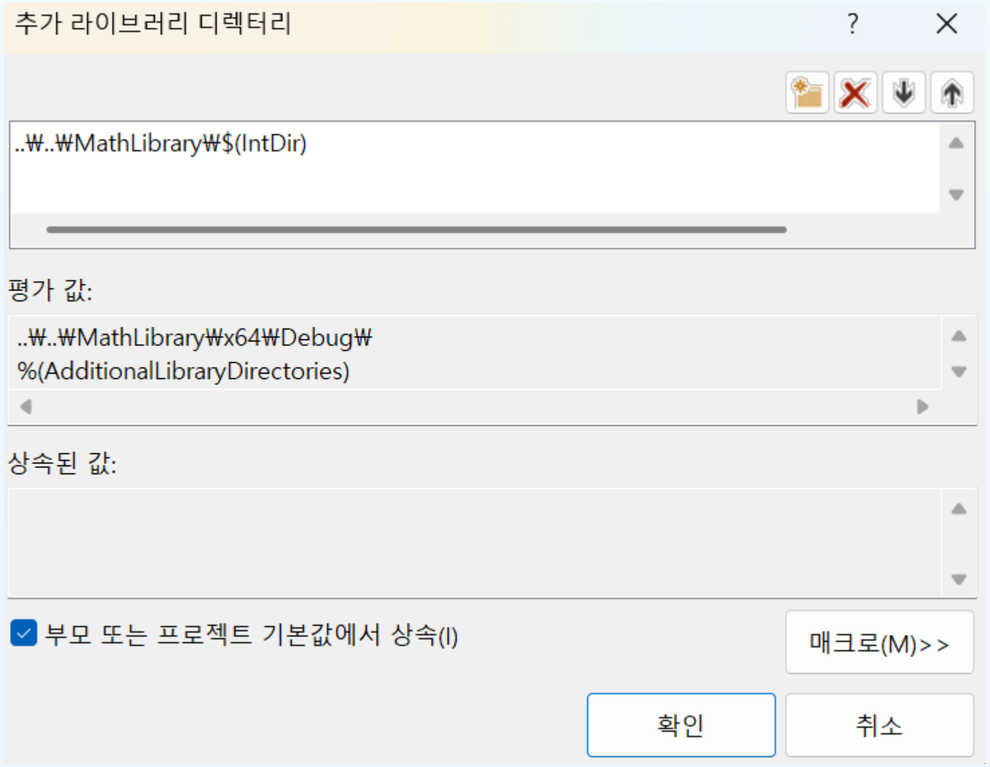



**상대 경로 및 매크로 방식 설정 전에 .lib 파일이 생성되는 위치를 중간 디렉터리인 $(IntDir) 경로로 지정하려면, 프로젝트 속성에서 링커의 가져오기 라이브러리 파일 경로를 직접 수정해주어야 합니다.**

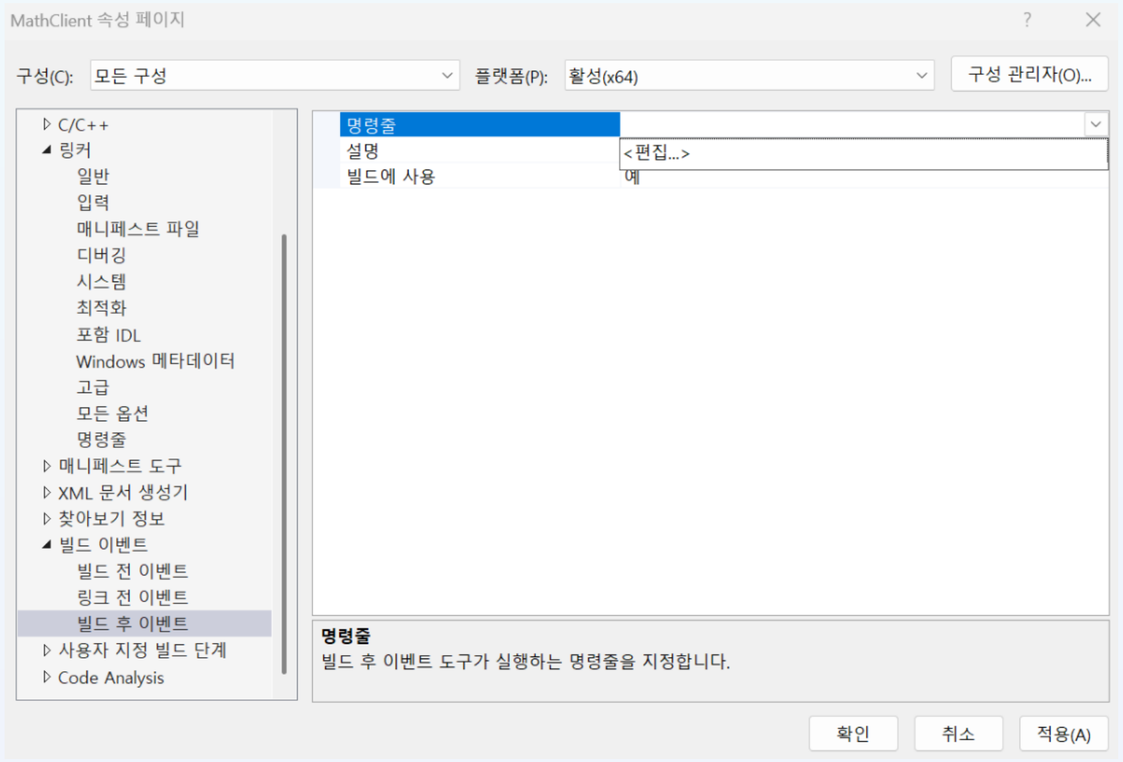
# 3. 프로젝트에 DLL 가져오기

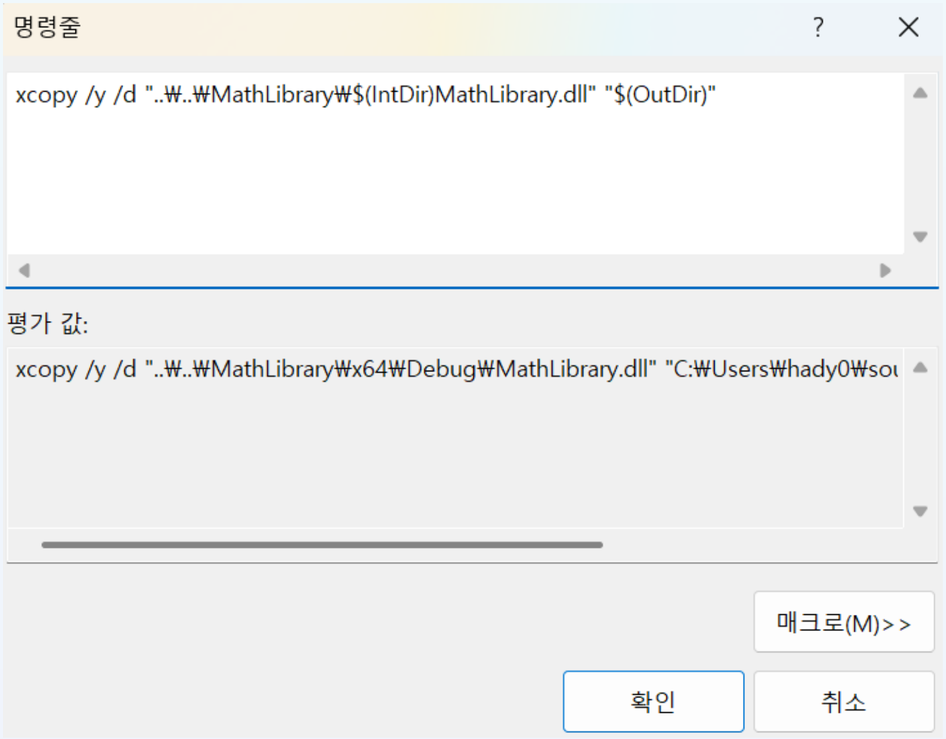



**..₩..₩MathLibrary₩$(IntDir) 명령어를 입력하여**
**DLL 및 클라이언트 프로젝트가 다른 위치에 있는 경우 일치하도록 상대 경로를 조정하는 이유는 빌드 구성(Configuration)이나 플랫폼이 바뀔 때 유연하게 대처할 수 있기 때문입니다.**

# 4. 빌드 후 이벤트 설정

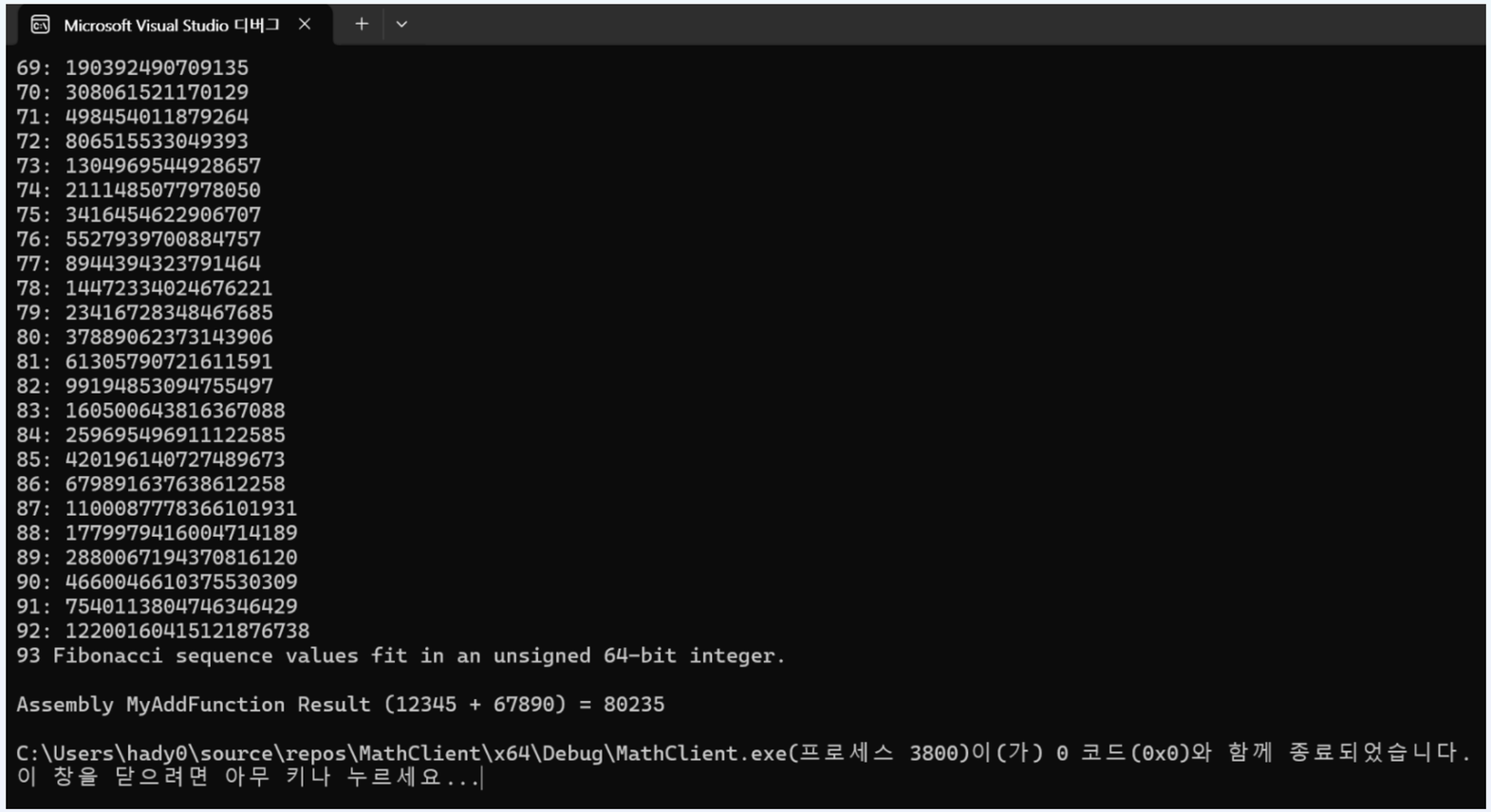



**xcopy /y /d "..₩..₩MathLibrary₩$(IntDir)MathLibrary.dll" "$(OutDir)" 명령어를 입력하여 DLL 및 클라이언트 프로젝트가 다른 디렉터리에 있는 경우 일치하도록 DLL의 상대 경로를 변경합니다.**

# 4. 클라이언트 앱 실행

```
1>------ 빌드 시작: 프로젝트: MathClient, 구성: Debug x64 ------
1>  MathClient.vcxproj -> C:₩Users₩hady0₩source₩repos₩MathClient₩x64₩Debug₩MathClient.exe
1>  0개 파일이 복사되었습니다.
========== 빌드: 1개 성공, 0개 실패, 0개 최신 상태, 0개 건너뜀 ==========
```




**빌드를 실행 후 성공하였다면 Ctrl + F5를 누르면 결과창이 뜹니다.**

# 감사합니다.